GUIA DE REPASO

Modulo 1

Data Analysis con Python
Clases 1 a 4

Una ayuda memoria para repasar fundamentos de Python, control de flujo, funciones, manejo de errores, list comprehensions y limpieza de strings.

Como usar esta guia

Lee primero la idea principal de cada clase. Usa las tablas para recordar la funcion de cada comando y los bloques de codigo como modelos para adaptar en los ejercicios.

Clase 1 - Introduccion al analisis de datos y Python

El analisis de datos transforma informacion cruda en respuestas concretas. El flujo habitual es obtener los datos, limpiarlos y transformarlos, analizarlos y comunicar los resultados.

Entorno de trabajo

Jupyter Notebook permite trabajar por celdas: ejecutas una parte del codigo, ves el resultado y continuas. El atajo mas usado es **Shift + Enter**: ejecuta la celda actual y pasa a la siguiente.

```
print("Hola, mundo del analisis de datos")
```

Git y GitHub

Git guarda el historial de cambios en tu computadora. GitHub aloja ese historial en internet. El flujo basico consiste en preparar los cambios, crear un commit y subirlo.

Comando	Para que sirve
git status	Muestra los archivos modificados y si hay cambios pendientes.
git add archivo	Prepara un archivo para incluirlo en el proximo commit.
git commit -m "mensaje"	Guarda una version local identificada por un mensaje.
git push	Sube los commits locales al repositorio remoto.
git clone URL	Descarga una copia local de un repositorio existente.

```
git add modulo-1-python/clase-01/practica/notebook.ipynb
git commit -m "Resuelve practica de la clase 1"
git push
```

Variables y tipos de datos

```
nombre = "Ana"
cantidad = 42
precio = 15990.50
es_premium = True
```

Tipo	Que representa	Ejemplo
str	Texto	"Notebook"
int	Numero entero	42
float	Numero con decimales	15990.50
bool	Verdadero o falso	True / False

Comandos y modelos utiles

Elemento	Funcion
print()	Muestra un resultado en pantalla.
type(valor)	Indica el tipo de dato de una variable.
int(), float(), str()	Convierten un valor a entero, decimal o texto.

Elemento	Funcion
f"...{variable}..."	Inserta valores dentro de un texto.
:,.2f	Muestra miles separados y dos decimales.

```
precio_texto = "1500"
precio = int(precio_texto)
print(f"Precio final: ${precio:,.2f}")
```

Recorda: los datos externos pueden llegar como texto. Antes de calcular, revisa su tipo y conviertelos si hace falta.

Clase 2 - Control de flujo y estructuras de datos

Esta clase permite que el programa tome decisiones, recorra colecciones de datos y acumule resultados. Es la base para analizar muchas filas sin repetir codigo manualmente.

Condicionales

```
if monto < 1000:
categoria = "micro"
elif monto < 100000:
categoria = "importante"
else:
categoria = "alto valor"
```

Palabra	Funcion
if	Ejecuta un bloque si la condicion es verdadera.
elif	Prueba otra condicion si las anteriores fueron falsas.
else	Define el caso restante.
and / or / not	Combina o niega condiciones.

Bucles y acumuladores

```
total = 0
for venta in ventas:
total += venta

promedio = total / len(ventas)
```

Elemento	Funcion
for	Recorre cada elemento de una coleccion.
while	Repite mientras una condicion sea verdadera.
range(5)	Genera 0, 1, 2, 3 y 4.
break	Sale del bucle inmediatamente.
continue	Salta al inicio de la siguiente vuelta.
+=	Suma y guarda el nuevo valor en una variable.

Listas

```
productos = ["notebook", "mouse", "teclado"]
productos.append("monitor")
primer_producto = productos[0]
ultimo_producto = productos[-1]
```

Herramienta	Funcion
lista[indice]	Accede a un elemento; el primer indice es 0.
.append(valor)	Agrega al final.

Herramienta	Funcion
.insert(posicion, valor)	Inserta en una posicion.
.remove(valor) / .pop()	Elimina un elemento.
len(), sum(), min(), max(), sorted()	Cuentan, calculan u ordenan listas.

Diccionarios

```
cliente = {"nombre": "Ana", "pais": "AR"}
print(cliente["nombre"])
telefono = cliente.get("telefono", "no registrado")
```

Un diccionario guarda pares clave-valor. Usa **.get(clave, valor_por_defecto)** cuando la clave podria no existir: evita un **KeyError**.

```
for clave, valor in cliente.items():
print(f"{clave}: {valor}")
```

Recorda: inicializa los acumuladores antes del bucle; las lineas indentadas se ejecutan en cada vuelta.

Clase 3 - Funciones y errores

Las funciones evitan repetir logica y hacen que el codigo sea mas facil de modificar. El manejo de errores permite procesar datos imperfectos sin detener todo el programa.

Funciones

```
def calcular_precio_final(precio, tasa_impuesto=0.19):
return precio * (1 + tasa_impuesto)

total = calcular_precio_final(320000)
```

Elemento	Funcion
def	Define una funcion.
parametros	Datos que la funcion recibe para trabajar.
valor=...	Define un parametro con valor por defecto.
return	Devuelve el resultado de la funcion.
pass	Marcador temporal: no ejecuta ninguna accion.

Una funcion puede devolver varios valores. Se reciben en el mismo orden.

```
def calcular_orden(precio, cantidad, descuento=0):
subtotal = precio * cantidad
monto_descuento = subtotal * descuento
return subtotal, monto_descuento, subtotal - monto_descuento

subtotal, descuento, total = calcular_orden(8500, 5, 0.10)
```

Manejo de errores

```
def convertir_temperatura(valor):
try:
return float(valor)
except ValueError:
return None
```

Concepto	Funcion
try	Intenta ejecutar un bloque que podria fallar.
except	Indica que hacer cuando ocurre una excepcion.
raise	Lanza manualmente un error con un mensaje.
else	Se ejecuta si no hubo error en try.
finally	Se ejecuta siempre.

Error	Caso tipico
ValueError	float("N/A") o una conversion invalida.
KeyError	Una clave no existe en un diccionario.
TypeError	Se combinan tipos incompatibles.

Error	Caso tipico
ZeroDivisionError	Se divide por cero.
IndexError	Se intenta usar una posicion inexistente.

Diseno modular

Una funcion deberia tener una sola responsabilidad: validar, calcular, agrupar o coordinar. Una funcion principal puede llamar a las pequenas para armar un pipeline claro.

Recorda: las variables creadas dentro de una funcion son locales. Para usarlas afuera, devuelve el resultado con return.

Clase 4 - List comprehensions y limpieza de strings

Una list comprehension crea una lista nueva transformando o filtrando los elementos de otra. La limpieza de strings prepara texto sucio para que pueda analizarse o convertirse a numero.

Sintaxis de list comprehensions

```
[expresion for elemento in lista]
[expresion for elemento in lista if condicion]
```

```
precios_con_iva = [precio * 1.19 for precio in precios]
precios_altos = [precio for precio in precios if precio > 20000]
```

Parte	Pregunta que responde
expresion	Que quiero guardar en la lista nueva?
for elemento in lista	Que lista voy a recorrer?
if condicion	Que elementos deben entrar?

Usa una comprehension cuando la operacion es corta y se entiende de un vistazo. Si hay muchas condiciones o decisiones, un bucle for tradicional suele ser mas legible.

Metodos de limpieza de texto

Metodo	Funcion	Ejemplo
.strip()	Quita espacios al inicio y al final.	" Ana ".strip() -> "Ana"
.replace(a, b)	Reemplaza un fragmento de texto.	"$1,200".replace("$", "")
.lower()	Convierte a minusculas.	"ANA".lower() -> "ana"
.capitalize()	Primera letra en mayuscula.	"mUEBLES".capitalize() -> "Muebles"
.title()	Primera letra de cada palabra en mayuscula.	"ana torres".title()

Limpiar y convertir precios

```
def limpiar_precio_usa(valor):
texto = valor.strip().replace("$", "").replace(",", "")
try:
return float(texto)
except ValueError:
return None
```

Para el formato estadounidense, la coma separa miles y se elimina; el punto se conserva porque es decimal. Antes de limpiar precios, confirma siempre el formato de origen.

```
precios_procesados = [limpiar_precio_usa(p) for p in precios_crudos]
precios_validos = [p for p in precios_procesados if p is not None]
```

Ejemplo de limpieza de emails

```
emails_limpios = [email.strip().lower() for email in emails_crudos]
emails_validos = [email for email in emails_limpios if email != "" and "@" in email]
```

Recorda: no filtres usando solo if precio, porque un valor 0.0 es valido. Para descartar conversiones fallidas usa if precio is not None.

Repaso final del Modulo 1

Este modulo construye la base para trabajar con datos antes de usar librerias como pandas. Primero se guarda y convierte informacion; luego se decide, recorre, valida, agrupa y limpia.

Mapa de decisiones rapido

Si necesitas...	Usa...
Guardar un dato	Una variable.
Ver el tipo de un dato	type().
Convertir texto a numero	int() o float().
Mostrar datos dentro de un texto	f-strings.
Tomar una decision	if / elif / else.
Recorrer datos	for.
Guardar varios elementos	Una lista [].
Guardar campos con nombre	Un diccionario {}.
Evitar que una conversion detenga el programa	try / except.
Reutilizar una operacion	Una funcion con def.
Crear una lista transformada o filtrada	List comprehension.
Limpiar texto	strip(), replace(), lower(), title().

Checklist antes de dar por terminado un ejercicio

1. Ejecuta la celda y lee el resultado.
2. Comprueba que los nombres de las variables sean claros.
3. Revisa la indentacion dentro de if, for, while, try y funciones.
4. Si usas datos externos, verifica tipos y valores faltantes.
5. Guarda el notebook con Ctrl + S.
6. Registra el avance en Git con git add, git commit y git push.

Modelo de cierre de practica

```
git status
git add modulo-1-python/clase-XX/practica/notebook.ipynb
git commit -m "Resuelve practica de la clase XX"
git push
```

Fin de la guia. Usa los ejemplos como referencia, pero intenta primero resolver cada consigna con tus propias palabras y despues compara tu idea con el modelo.